

=====

Python Function Complete Task

=====

1. What is a Function?

A Function is a block of code that performs a specific task.

Instead of writing the same code again and again,

we put it inside a function and call it whenever needed.

1) Simple Function :

```
In [4]: def my_name():      # my_name ..is a function name
        print("Azim")
        my_name()          # call the function
```

Azim

```
In [2]: def greet():
        print("Hello Good Evening")
        greet()
```

Hello Good Evening

```
In [3]: def greet():
        print("Hello Good Morning")
        greet()

        def greet():
            print("Hello Good Morning")
            greet()

        def greet():
            print("Hello Good Morning")
            greet()
```

Hello Good Morning
Hello Good Morning
Hello Good Morning

```
In [4]: # insted of this we can make easier and short code like as below

        def greet():
            print("Hello Good Morning")
```

```
greet()  
greet()  
greet()  
greet()
```

Hello Good Morning
Hello Good Morning
Hello Good Morning
Hello Good Morning

2) Function with No Argument and No Return

```
In [5]: def welcome():  
        print('Welcome to Nit')  
        welcome()
```

Welcome to Nit

3) Function With One Argument

```
In [11]: def student(name):    # name is argument(1)  
        print("Student name is :", name)  
  
        student("Azim")  
        student("Amir")  
        student("SRK")    # we can add multiple objects
```

Student name is : Azim
Student name is : Amir
Student name is : SRK

4) Function with Two Arguments

```
In [8]: def add(a,b):  
        print("add =", a+b)  
  
        add(10, 20)  
        add(22, 13)
```

add = 30
add = 35

```
In [9]: def sub(a,b):  
        print("add =", a-b)    # here function only observe on '-' arithmetic sign on  
  
        sub(10, 20)  
        sub(22, 13)
```

```
add = -10  
add = 9
```

5) Function with Return

```
In [13]: def squre(num):  
         return num*num  
  
         Result= squre(5)  
  
         print(Result)
```

25

```
In [15]: def sub(num):  
         return num-num  
  
         result = sub(5)    # here if we have add 1 attribute then in this call bracet also  
  
         print(result)
```

0

```
In [16]: def add(num):  
         return num+num  
  
         result = add(50)  
  
         print(result)
```

100

```
In [20]: def squre(name):  
         return name*name  
  
         output = squre(2)  
  
         print(output)    # return just focus on sign of arthematic, and give result back to
```

4

```
In [10]: def add(num):  
         return num+30*2  
  
         result = add(50)  
  
         print(result)    # return only produce output which is it contain .
```

110

```
In [9]: def num():  
         for i in range(10):  
             return i
```

```
print(num())
```

0

```
In [29]: def add(num):  
         return 5  
  
         result = add(50)  
  
         print(result)  # it give thus value which is in return function have.
```

5

6) Difference between print and return

```
In [18]: # using print  
  
def add():  
    print(10+20)  
  
a = add()  # here return can only give output one time after we print seconde tim  
print(a)
```

30

None

```
In [16]: # Using return  
def add():  
  
    return 10+20  
  
a = add()  
  
print(a)  # using return it give value which is have.
```

30

7) Multiple Return Values

```
In [22]: def student():  
         return 'Azim', 25, 'Solapur'  
  
         name,age,city = student()  # call the all function  
  
         print(name)  
         print(age)  
         print(city)
```

Azim

25

Solapur

8) Positional Arguments

```
In [27]: def emp(name, salary):
          print(salary, name)

          emp("Azim", 50000)    # here in print, whatever we put in 1st position it display tho
                                # In def () 1st name argument mention then in call function emp

50000 Azim
```

```
In [30]: def std(name, ID):

          print(ID)
          print(name)

          std("Azim", 101)
          std( 44, "Ajim")    # here we add wrong position but in def 1st is name argument the
                                # then 44 name is print in 2nd position as we mention in print()

101
Azim
Ajim
44
```

```
In [32]: def std(name, age):
          print(age+5)
          print(name*2)

          std(" Ajim "*5, 25-2)    # here we also changes in the value in print section
                                    # if in print we add 1 st age then it print 1st age argume
                                    # here we also use this some changes in object variable we

28
Ajim Ajim Ajim Ajim Ajim Ajim Ajim Ajim Ajim Ajim
```

9) Keyword Arguments

```
In [35]: def emp(name, salary):
          print(name)
          print(salary)

          emp(salary = 50000, name = "Ajim")    # here we added keyword.
          emp(salary= "raj", name = 10000)

          # here order does not matter.
          # if we add 1st keyword name then he give output name is in 1st position.
          # in this 2nd object call functionn we mention salary = raj ,
          # then in print section salary is in 2nd position, hence salary = raj in 2nd posi
```

```
Ajim
50000
10000
raj
```

```
In [70]: def std(name,age, roll_no):
          print(name,age,roll_no)

          std(age = 23, roll_no = 21, name = "Rama")
          std(roll_no = 44, name = "Aj", age = 25)  # position is matter..
```

```
Rama 23 21
Aj 25 44
```

```
In [19]: def emp(name,salary):
          print(name)
          print(salary-5000)

          emp(salary = 50000+10000, name = "Ajim")  # here we added keyword.
          emp(name = "raj", salary = 10000)

          # here we also use this some changes in print section and also in object variable w
```

```
Ajim
55000
raj
5000
```

10) Positional vs Keyword Arguments

```
In [42]: # Positinal
          def std(name,age):
              print(name, age)

          std(22,"Aj")  # here python consider 22 is name and aj is age, hence as per print(
          std("Pk",11)
          std(31, "MM")

          # keyword
          std(age = 22, name = "Aj")  # In print section name in 1st position hence name = aj
          std(name = "Sk", age = 33)  # Now order does not matter only position is important,
          std(age = "MM", name = 25)
```

```
22 Aj
Pk 11
31 MM
Aj 22
Sk 33
25 MM
```

11) Default Arguments

```
In [80]: def drink(item = "Tea"):    # we make Tea is a default.
        print(item)

        drink()    # if here does not put any value then he auto fill with default val
        drink("coffee")    # If we change diff. arg. then it display what we mention.
```

Tea
coffee

```
In [78]: def drink(name, item = "Tea"): # here we make tea as a default, 2 argument use
        print(name,item)

        drink("Azim","coffee")    # here we add different arg. then default item does not co
        drink("Lemon_Tea","Aj")    # here python consider lemon tea is a name and aj is item,
        drink("Tea")                # here we write name is tea then automatically give tea as a
        drink("Ajim")                # here we write name then default arg comes after that.
```

Azim coffee
Lemon_Tea Aj
Tea Tea
Ajim Tea

12) Variable Length Arguments(*args)

```
In [6]: # without length argument
def total(a,b):
    print(a+b)

total(10,20)    #.. It works because 2 argument, 2 variables
total(10,20,30)    # ..error because here 3 variable it does not allow
```

30

```
-----
TypeError                                Traceback (most recent call last)
Cell In[6], line 7
      3     print(a+b)
      6 total(10,20)    #.. It works because 2 argument, 2 variables
----> 7 total(10,20,30)    # ..error because here 3 variable it does not allow

TypeError: total() takes 2 positional arguments but 3 were given
```

```
In [57]: # * = "Collect all positional arguments into one tuple."
def total (*numbers):    # Add more amount of numbers.
    print(sum(numbers))    # here we use only sum keyword (numbers is tuple)and nothi

total(10,30)    # python add all element.
total(10,30,40)
total(10,20,30,40,)
```

40
80
100

```
In [9]: def total (*nums):  # * is important not name .
        print(sum(nums))
        print(type(total))  # type is function
        print(type(nums))   # type is tuple.

        total(10,30)
        total(10,30+40)
        total(10,20,30*40,)
        total(50-40,300*2,100/25,10%100)  # here it calculate all then finally take sum of

40
<class 'function'>
<class 'tuple'>
80
<class 'function'>
<class 'tuple'>
1230
<class 'function'>
<class 'tuple'>
624.0
<class 'function'>
<class 'tuple'>
```

13) Loop Through *args

```
In [46]: def display(*items):

        for item in items:
            print(item)

        display("Pen", "Book", "Laptop")
```

Pen
Book
Laptop

```
In [49]: def display(*details):

        for item in details:
            print(item*2)

        display("Water bottel", "School Bag", "Laptop")
```

Water bottelWater bottel
School BagSchool Bag
LaptopLaptop

Add Numbers Without sum()

```
In [55]: def total(*args):
        result = 5  #...5 = num
        count = 2
```



```

    for num in args:
        result = result + num
        count += 1

    print(result)
    print(count)

total(10, 20, 30, 40) # it display total result of this object then add 5 as we me

```

105
6

14) Keyword Variable Argument(**kwargs)

```

In [2]: def student(**kwargs):    # store name = value pairs.
        print(kwargs)
        print(type(kwargs))    # type is dict.

        student(name="Azim", age = 22)

```

```

{'name': 'Azim', 'age': 22}
<class 'dict'>

```

```

In [17]: def employee(**details):

        print(details)
        print(type(details))

        employee(emp_name = "Rajesh",
                  emp_id = 101,
                  emp_salary = 40000,
                  emp_city = "Hyd")

```

```

{'emp_name': 'Rajesh', 'emp_id': 101, 'emp_salary': 40000, 'emp_city': 'Hyd'}
<class 'dict'>

```

15) Local Variable

```

In [22]: def data():
        a = 1000 # here a value is written after def function
        print(a)

        data() #data() takes 0 positional arguments but 1 was given

```

1000

16) Global Variable

```

In [23]: a = 500 # here a value is written before def function
        def data():
            print(a)

```

```
data()
```

500

Global variable vs Local Variable

- We discussed about variable & discussed about function
- Sometimes we are creating variable inside the function and sometimes we are creating variable outside of the function
- when we declare variable outside of the function that concept called as scope

```
In [12]: a = 10
         print(a)
```

10

```
In [15]: a = 10    # global variable

def something():
    b = 15    # local variable
    print('in function', b)
    print('out function', a)

# in this code we are declaring 2 variable is this possible
# first line of a is called outside of the function
# inside the function is called local variable
```

```
In [16]: a = 10

def something():
    a = 15

print('in function', a)

print('out function', a)

# in this code we are declaring 2 variable is this possible
# first line of a is called outside of the function
# inside the function is called local variable
```

```
in function 10
out function 10
```

```
In [19]: a = 10

def something():

    a = 15    # here a is local variable
    b = 8

    print(a)
```

```
print(b)
print(a)
```

```
In [20]: a = 10
def something():
    a = 15    # hear a is local variable
    print(a)
    print(a)
```

```
In [14]: a = 10

def something():
    b = 15
    print('in function',b)

print('out function',a)

# in this code we are declaring 2 variable is this possible
# first line of a is called outside of the function
# inside the function is called local variable
```

out function 10

```
In [21]: a = 10

def something():
    #if we remove this variable then can be default it consider as global variable
    print('in function',a)

something()
print('out function',a)
# if we dont assign any variable inside the function by default both considered as local
```

in function 10
out function 10

Pass List to Function

- can we pass list of element in the function that function will return the count of even or odd number from the list

```
In [22]: def count(lst):

    even = 0
    odd = 0

    for i in lst:
        if i%2 == 0:
            even += 1
        else:
            odd += 1
    return even,odd

lst = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10,11, 12, 20]
```

```
even, odd = count(lst)

print(even)
print(odd)
```

7
6

In [23]: **def** count(lst):

```
    even = 0
    odd = 0

    for i in lst:
        if i%2 == 0:
            even += 1
        else:
            odd +=1
    return even,odd
```

```
lst = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10,11]
even,odd = count(lst)
```

```
print("Even Number: {} and odd Number : {}".format(even,odd))
#format is function belongs to string & bydefault you need to pass any parameter
```

Even Number: 5 and odd Number : 6

Fibonacci Sequence

- As fib is not inbuilt function so we need to define the function for it

In [24]: **def** fib(n):

```
    print(0)
    print(1)
```

```
fib(0)
```

```
# in the above code we can get the fibonacci series but if the number is
# large then it takes more time
```

0
1

In [25]: **def** fib(n):

```
    print(0)
    print(1)
    print(1)
    print(2)
    print(3)
    print(5)
```

```
fib(0)
```

0
1
1
2
3
5

In [26]: *# in programmin we need to continue these process thats why we need to use loop hear*

```
def fib(n):
    a = 0
    b = 1

    print(a)
    print(b)

    for i in range(2, n):
        c = a + b
        a = b
        b = c

        print(c)

fib(10)
```

0
1
1
2
3
5
8
13
21
34

In [27]: *# Ignore below code*

*'''if user wants 5 value then above code is applicable but if user wants only 1 val
#fib(1) then you will get 2 vales thats why we need to write the condition hear.'''*

```
def fib(n):
    a, b = 0, 1
    if n == 1:
        print(a)
    else:
        print(a)
        print(b)

    for i in range(2, n):
        c = a + b
        a = b
        b = c
        print(c)

fib(2)
```

0
1

Factorial of a Number in Python

- $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$
- $5! = 120$

```
In [28]: def fact(n):
          f = 1
          for i in range(1, n+1):
              f = f*i

          return f

          x = 5
          result = fact(x)
          print(result)

          # please use debug the code in pycharm for more indetail explanation &
          # breakthrough point at f = 1
```

120

In []:

17) Lambda Add

```
In [29]: # 1st Normal function
          def squre(x):

              return x * x

          print(squre(6))
```

36

```
In [37]: # By using lambdawe get same output with Less code.
          add = lambda a,b : a+b

          print(add(10,5))
```

15

Lamba with one Argument

```
In [39]: cube = lambda x: x**3

          print(cube(2))
```

8

Lambda with Multiple Arguments

```
In [42]: average = lambda a, b, c : (a+b+c)/3  
  
         print(average(11,22,33))
```

22.0

Lambda with If-Else

```
In [46]: largest = lambda a,b : a if a>b else b  
  
         print(largest(40,10))
```

40

Lambda with map()

```
In [49]: # 1st Without Lambda:  
  
         nums = [1,2,3,4]  
  
         result = []  
  
         for n in nums:  
             result.append(n*n)  
  
         print(result)
```

[1, 4, 9, 16]

```
In [50]: # by using lambda:  
  
         nums = [1,2,3,4]  
  
         result = list(map(lambda x: x*x, nums))  
  
         print(result)
```

[1, 4, 9, 16]

Lambda with filter()

```
In [52]: nums = [1,2,3,4,5,6,7,8,9,10]  
  
         even = list(filter(lambda x : x%2 == 0, nums))  # here %2== 0 means reminder need  
                                                         # when we devide any value with 2,  
  
         print(even)
```

[2, 4, 6, 8, 10]

Lambda with sorted()

```
In [ ]: students = [
    ('Ajay', 80),
    ('Ali', 75),
    ('Amir', 90)
]
result = sorted(
    students,          # Lambda is Used only once.
    key = lambda x : x[1] # here 1 means is select 1 position value in 1st.(0=ajay)
)

print(result)
# Use the second value (marks) for sorting.
```

- What ever even number I have from the assigned list
- I want to double the even number i.e 2 become 4 || 4 become 6 || 6 become 8
- that we will do using map function
- this largely we are using in google map reduce programme
- we can build using user define & lambda

```
In [36]: def is_even(n):
    return n % 2 == 0

nums = [3, 2, 6, 8, 4, 6, 2, 9]
evens = list(filter(is_even, nums))

print(evens)
```

[2, 6, 8, 4, 6, 2]

```
In [39]: def is_odd(n):
    return n % 2 != 0

nums = [3, 2, 6, 8, 4, 6, 2, 9]

odd = list(filter(is_odd, nums))
print(odd)
```

[3, 9]

```
In [37]: # Lets take one list & i want to find the list of even numbers

nums = [3, 2, 6, 8, 4, 6, 2, 9]

evens = list(filter(is_even, nums)) # is_even is not an inbuilt function
```

```
In [38]: def update(n):
    return n+2

nums = [3, 2, 6, 8, 4, 6, 2, 9]
```



```
evens = list(filter(is_even, nums))
double = list(map(update, evens))

print(double)
```

[4, 8, 10, 6, 8, 4]

In [40]: *# Lets write above function using help of Lambda & Lambda helps to reduce the line*

```
nums = [3,2,6,8,4,6,2,9]
evens = list(filter(lambda n : n%2 == 0, nums))
print(evens)
```

[2, 6, 8, 4, 6, 2]

In [41]:

```
nums = [3,2,6,8,4,6,2,9]
odd = list(filter(lambda n : n%2 !=0, nums))
print(odd)
```

[3, 9]

In [42]: *# Lets write above function using help of Lambda & Lambda helps to reduce the line*

```
nums = [3,2,6,8,4,6,2,9]

evens = list(filter(lambda n : n%2 ==0, nums))
odd = list(filter(lambda n : n%2 !=0, nums))

print(evens)
print(odd)
```

[2, 6, 8, 4, 6, 2]

[3, 9]

- What ever even number I have from the assigned list
- I want to double the even number i.e 2 become 4 || 4 become 6 || 6 become 8
- that we will do using map function
- this largely we are using in google map reduce programme
- we can build using user define & lambda

In [43]:

```
def update(n):
    return n+2

nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))
double = list(map(update, evens))

print(double)
```

[4, 8, 10, 6, 8, 4]

In [44]:

```
nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))
double = list(map(lambda n : n*2, evens))
```

```
print(double)
```

```
[4, 12, 16, 8, 12, 4]
```

```
In [45]: nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))
double = list(map(lambda n : n*2, evens))
double_ = list(map(lambda n : n+2, evens))

print(double)
print(double_)
```

```
[4, 12, 16, 8, 12, 4]
```

```
[4, 8, 10, 6, 8, 4]
```

```
In [46]: nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))

double = list(map(lambda n : n*2, evens))
double_ = list(map(lambda n : n-2, evens))

print(double)
print(double_)
```

```
[4, 12, 16, 8, 12, 4]
```

```
[0, 4, 6, 2, 4, 0]
```

```
In [47]: nums = [3,2,6,8,4,6,2,9]
evens = list(filter(is_even, nums))

double = list(map(lambda n : n*2, evens))
double_ = list(map(lambda n : n-2, evens))
double1 = list(map(lambda n : n+2, evens))

print(double)
print(double_)
print(double1)
```

```
[4, 12, 16, 8, 12, 4]
```

```
[0, 4, 6, 2, 4, 0]
```

```
[4, 8, 10, 6, 8, 4]
```

- i want to perform reduce now
- i want reduce all the values
- reduce you can add only 2 values
- [4, 12, 16, 8, 12, 4] if you sum everything then you will get 56

```
In [48]: from functools import reduce

def add_all(a,b):
    return a+b

nums = [3,2,6,8,4,6,2,9]
```

```
evens = list(filter(is_even, nums))
double = list(map(lambda n : n*2, evens))

sums = reduce(add_all, double)
sums
print(sums)
```

40

```
In [49]: from functools import reduce

nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))
double = list(map(lambda n : n*2, evens))
sums = (reduce(lambda a,b : a + b, double))

print(evens)
print(double)
print(sums)
```

```
[2, 6, 8, 4, 6, 2]
[4, 12, 16, 8, 12, 4]
56
```

In []:

18) Recursive Function

```
In [ ]: # It is a function that calls itself.
# A function repeatedly calls itself until a stopping condition is reached.
```

```
In [73]: def countdown(n):

    if n == 0:
        return 0    # here 0 is a stopping point hence return also be 0 or
                    # we write only single return also.

    print(n)

    countdown(n-1)

countdown(5)
# here in 1st,2,3,4 call is 5,4,3,2 hence condition is false is not = 0
# but in 5th call condition is satisfied is = 0 hence is true, then it will stop he
```

```
5
4
3
2
1
```

19) Using factorial recursive function:

```
In [72]: # using factorial recursive function:

def factorial(n):

    if n == 1:      # Think return 1 is like wall, we need to stop here.
        return 1   # 1! =1 , So when recursion reaches 1, we already know the answer
                    # "Stop recursion. The factorial of 1 is 1."

    return n * factorial(n-2) # here return 1 means it multiply the result by 1 at
                             # because our condition is satisfied n==1.

print(factorial(5))
```

15

20) task on student Result function

```
In [78]: def result (marks):
        if marks >= 35:
            return "Pass"
        return "Fail"

student_marks = int(input("Enter marks : "))

print(result(student_marks))
```

Pass

21) task on Employee details :

```
In [84]: def employee(salary):
        if salary <= 30000:
            return "Freshers Employee"
        return "Senior Employee"

employee_salary = int(input("Enter salary : "))

print(employee(employee_salary))
```

Freshers Employee

Python Decorators

This is one of amazing feature in python

We knew that function are build on certain task

lets understand we have some predefined function

div() function which takes 2 parameter

```
In [1]: def div(a,b):
        print(a/b)
        div(4,2)

        # but what if we pass the values 2, 4
```

2.0

```
In [2]: def div (a,b):
        print(a/b)
        div(2, 4)
```

0.5

- i want different logic hear
- it does not matter in which sequence i pass the value alway numerator should be greather then denomenator (numerator > denomenator)
- in the above function when i divide 2/4 (numerator<denomenator)
- even though user pass 2/4 but inbuild it should consider as 4 / 2 but how to swap lets discuss
- before division 2/4 lets apply the logic & logic is if 2<4 then i want to swap

```
In [3]: def div(a,b):
        if a<b:
            a,b = b,a
        print(a/b)
        div(2,4)
```

2.0

- lets understand the twist, imagine the code you dont have , you work in the organization
- inbuild the code & you dont want to change the existing code
- without impacting on existing code you need to get the solution
- being a developer you need to swap the value without touching the existing function
- thats the reason DECORATOR comes into the picture
- using the help of DECORATOR you can add the extra feature in the existing function
- lets create a new function for decorator
- we can write functin inside the funtion thats why we called as python is functional programing language

```
In [4]: # using help of the decorator you can add the extra feature in the exicting functio

def div(a,b):
    print(a / b)
```

```
def div_decorator(func): # hear div_dectorator will accept the div function
    def inner(a,b):
        if a<b:
            a,b = b,a
        return func(a,b)
    return inner

div = div_decorator(div)

div(2,4)
```

2.0

```
In [5]: def my_decorator(func):
        def wrapper():
            print("Something is happening before the function is called.")
            #func()
            print("Something is happening after the function is called.")
        return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

Something is happening before the function is called.

Something is happening after the function is called.

```
In [6]: def my_decorator(func):
        def wrapper():
            print("Something is happening before the function is called.")
            func()
            print("Something is happening after the function is called.")
        return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

Something is happening before the function is called.

Hello!

Something is happening after the function is called.

- the best part of the decorator is you can change the behaviour of existing function at the time of compiler
- what we done now we take new function which take func as an parameter
- you cant do like this any programming language
- we can defin the function with in the function which actually replacing using swap
- without changing existing code with the help of decorator we are get the output

Modules

- concept of debugging means - remove the bugs
- what is coding - coding will create bugs or add the bugs
- Breakdown complex codes or logical part to different modules
- if you see the image
- let's break down to logical part as is 1 module & bd is another module
- again we are breaking into c module, a module, b module, d module as well
- main advantage is if you change module a it won't effect on other module b, or c
- another advantage is you can re-use the module
- if you build other module as GADP (A & D) module is reused
- Inbuilt module (import numpy as np)
- user define module as well (let's create how to define user define module)
- for that let's take one new file as test.ipynb which define 4 function (add, sub, multi, div)
- i want to use this function in another module
- so let's create our own module
- please use spyder.py for practice which saved in datascience project
- let's create a def function with add, sub, multiple, divisible & stored under test.py
- in order to call then in separate module called calc.py

Special Variable name

```
In [7]: __name__
```

```
Out[7]: '__main__'
```

```
In [10]: print(__name__)
__main__
```

- The moment when we print **name** then i will get output as main
- in your project if you have 5 modules or 10 modules there will some modules which will be run first

- so the first module name is always **main**. that's why the code starts
- to understand in detail review we will work with PYCHARM
- from the main function we can call all other functions
- please work 4 files in pycharm (hi.py, bye.py, hi1-specialvariable, bye1-specialvariable)
- refer more about chatgpt

In []:

In []: